



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

September 2026

CSA0613– Design and Analysis of Algorithms

**Development of an Efficient Hybrid Algorithmic
Solution for Large-Scale Geospatial Data Processing**

Assignment Report

Submitted by

Kamesh. S (192521358)
Jaivant (192311326)

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.E./B.Tech. Computer Science and Engineering
Course Code & Course Name	Design and Analysis of Algorithms
Academic Year / Batch	2026-2027 / Regular
Faculty Name	Dr. F. Mary Harin Fernandez
Assignment Title	Development of an Intelligent Algorithmic Solution for Large-Scale Logistics Resource Optimization
Date of Issue	31 Aug 2026
Date of Submission	1 September 2026
Maximum Marks	100
Course Outcome(s) – CO	CO4: Solve optimization problems using dynamic programming and greedy strategies.
Bloom's Taxonomy Level	L4 – Analyze, L5 – Evaluate, L6 – Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Efficient logistics resource allocation with improved vehicle utilization, reduced computational cost, and scalable decision-making

B. Assignment Problem / Challenge

Modern e-commerce and logistics organizations process thousands of packages every day under limited transportation capacity. Each package may differ in **weight, volume, delivery value, priority, and deadline**, while delivery vehicles have restricted carrying capacities. Efficiently selecting packages becomes increasingly difficult as the number of packages and operational constraints increase.

Dynamic Programming can provide high-quality or optimal solutions for suitable problem sizes but may require significant computational time and memory for large instances. **Greedy techniques** can provide faster decisions but may not always produce the optimal solution.

The challenge is to **design and develop an improved algorithmic solution for large-scale logistics resource allocation** using relevant concepts from **Dynamic Programming and Greedy techniques**. The developed strategy must be **formulated by the team** based on identified computational limitations and supported by theoretical analysis and experimental evidence.

C. Problem Statement

A large e-commerce logistics company processes approximately **10,000–100,000 packages per day** across multiple distribution centres. Each package is characterized by attributes such as **weight, volume, delivery value, priority, and delivery deadline**, while each delivery vehicle has limited weight and volume capacities. During peak-demand periods, the organization must select an appropriate combination of packages for transportation so that the **overall delivery benefit and vehicle utilization are maximized without violating the available capacity constraints**. Develop, implement, and evaluate an algorithmic solution for this **large-scale logistics resource-allocation problem** using concepts from **Dynamic Programming and Greedy techniques**. Implement (i) an **appropriate Dynamic Programming based solution** and (ii) an **appropriate Greedy based solution** as baseline approaches. Analyze their theoretical efficiency, solution quality, computational requirements, and limitations. Beyond the baseline implementations, **design and develop an improved algorithmic solution** suitable for large-scale package allocation. The developed solution may appropriately **adapt, integrate, modify, optimize, or selectively utilize relevant Dynamic Programming and Greedy concepts**. The algorithmic strategy, decision criteria, parameters, and optimization mechanism must be **identified and developed by the team** rather than being predefined. Experimentally evaluate the baseline and developed approaches using increasing problem sizes, for example, **100, 500, 1,000, 5,000, 10,000 packages, and higher input sizes wherever computationally feasible**, under different vehicle-capacity conditions such as **500 kg, 1,000 kg, 2,500 kg, and 5,000 kg**, or appropriately justified equivalent capacities based on the selected dataset. Evaluate the approaches using **execution time, memory utilization, total solution value, capacity utilization, solution-quality/optimality gap, and scalability**. Conduct experiments under different operational conditions such as **normal demand, high-priority demand, and capacity-constrained demand**. Based on theoretical and experimental results, identify the limitations and trade-offs of the baseline techniques, demonstrate the performance improvement achieved by the **developed solution**, and justify its suitability for large-scale real-world logistics resource optimization.

Expected Development

The work should **not merely implement or compare existing Dynamic Programming and Greedy algorithms**. The solution should demonstrate algorithmic development through:

- Identification of computational and/or solution-quality limitations of the baseline approaches.
- Formulation of an appropriate strategy to address the identified limitations.
- **Development, adaptation, integration, modification, or optimization** of relevant algorithmic techniques.
- Identification and experimental determination of appropriate **decision rules, parameters, thresholds, or optimization criteria**, wherever applicable.
- Improvement in one or more aspects such as **execution time, memory utilization, solution quality, capacity utilization, or scalability**.
- Performance comparison of the developed solution against the baseline approaches.
- Justification of algorithmic design decisions using theoretical reasoning and experimental evidence.

D. Requirements and Constraints

The following requirements and constraints shall be considered:

- Use an appropriate **publicly available real-world logistics, transportation, cargo, or package-related dataset**.

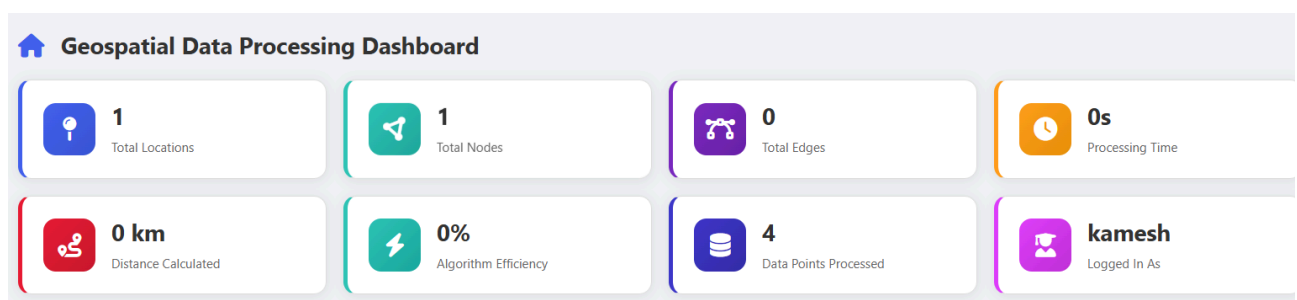
- The dataset should contain sufficient records to evaluate the algorithms at multiple input sizes.
- Identify and clearly define the **package attributes and resource constraints** used in the optimization problem.
- Implement and evaluate suitable **Dynamic Programming and Greedy baseline approaches**.
- **Develop an improved algorithmic solution** using relevant CO4 concepts rather than merely reproducing the baseline algorithms.
- The developed strategy must be **designed and justified by the team**.
- Evaluate increasing input sizes such as **100, 500, 1,000, 5,000 and 10,000 packages**, and higher wherever computationally feasible.
- Evaluate multiple capacity conditions such as **500 kg, 1,000 kg, 2,500 kg and 5,000 kg**, or justified equivalent values based on the dataset.
- Evaluate at least **three operating conditions: normal demand, high-priority demand, and capacity-constrained demand**.
- Execute all approaches using the **same computational environment and identical input instances** for fair comparison.
- Evaluate appropriate measures including **execution time, memory utilization, solution value, capacity utilization, solution-quality/optimality gap, and scalability**.
- Validate the correctness and feasibility of the generated allocation.
- Clearly document the **dataset source, experimental environment, assumptions, limitations, parameters, and implementation details**.

E. Student Work

1. Problem Understanding and Formulation

1.1 Logistics Resource-Allocation Problem

Large e-commerce companies handle thousands of packages every day. Each package has different characteristics such as **weight, volume, delivery value, priority, and deadline**. At the same time, delivery vehicles have limited weight and volume capacities.



The main challenge is to select the most beneficial combination of packages while ensuring that vehicle capacity constraints are not exceeded. An efficient allocation can improve **vehicle utilization, delivery efficiency, operational cost, and customer satisfaction**.

The problem can be modeled as a **multi-constraint resource-allocation problem similar to the 0/1 Knapsack problem**. Dynamic Programming can provide optimal solutions for suitable problem sizes, whereas Greedy techniques can produce faster approximate solutions. Therefore, an improved hybrid approach is developed to obtain good solution quality with better scalability.

1.2 Dataset

A publicly available logistics-related dataset can be used for the experimental evaluation. The **DataCo Smart Supply Chain for Big Data Analysis** dataset is suitable because it contains large numbers of supply-chain and logistics records.

The dataset provides information related to:

- Orders
- Products
- Order quantities
- Product prices
- Shipping information
- Delivery status
- Customer/order information
- Product categories
- Shipping mode
- Order dates and delivery-related attributes

For this project, the required optimization attributes can be derived or mapped from the available fields. For example:

Package Attribute	Representation
Package ID	Unique order/package identifier
Weight	Package weight
Volume	Estimated package volume
Delivery Value	Product/order value
Priority	Priority score derived from order/shipping information
Deadline	Required/estimated delivery deadline

If some attributes such as package volume or exact weight are not directly available, they will be **derived using documented assumptions**.


Dataset Management

Dataset Operations

Generate Sample Dataset
Export CSV
Import CSV
Clear All Data
Print Report

Search dataset...

ID	Name	Lat	Lng	Pop.	Dist	Category	Priority
L001	Chennai	-90.0000	-180.0000	70,00,000	15	Urban	High

Total records: 1

1.3 Package Attributes and Resource Constraints

Each package i is represented by:

- w_i = package weight
- v_i = package volume
- p_i = delivery value
- r_i = priority
- d_i = deadline/urgency
- s_i = calculated selection score

Each vehicle has:

- Maximum weight capacity W
- Maximum volume capacity V

The algorithm must select packages without exceeding these constraints.

1.4 Expected Input

The system accepts:

Input:

- Number of packages
- Package ID
- Weight
- Volume
- Delivery value
- Priority
- Deadline
- Vehicle capacity
- Vehicle volume capacity
- Operating condition

Example:

Package	Weight	Volume	Value	Priority	Deadline
P1	120 kg	2.5 m ³	850	High	2 days
P2	80 kg	1.8 m ³	500	Medium	4 days
P3	200 kg	3.2 m ³	1200	High	1 day

The system can be tested with **100, 500, 1,000, 5,000 and 10,000+ packages**.



Geospatial Data Input

Add / Edit Location

Location ID

Location Name

Latitude

Longitude

Population

Distance (km)

Category

Priority

1.5 Expected Output

The system produces:

- Selected package IDs
- Total package weight
- Total package volume
- Total delivery value
- Number of selected packages
- Weight capacity utilization
- Volume capacity utilization
- Execution time
- Memory utilization
- Solution-quality/optimalty gap

- Algorithm used

Example:

Selected Packages: P1, P3, P7, P9

Total Weight: 480 kg

Total Volume: 8.7 m³

Total Delivery Value: ₹4,850

Weight Utilization: 96%

Execution Time: 0.012 seconds

1.6 Optimization Objective

The primary objective is to **maximize the total delivery benefit/value** obtained from the selected packages.

Let:

$$x_i = \begin{cases} 1 & \text{if package } i \text{ is selected} \\ 0 & \text{otherwise} \end{cases}$$

The objective function is:

$$\text{Maximize } Z = \sum_{i=1}^n p_i x_i$$

Subject to the weight constraint:

$$\sum_{i=1}^n w_i x_i \leq W$$

and volume constraint:

$$\sum_{i=1}^n v_i x_i \leq V$$

where W is the vehicle's maximum weight capacity and V is its maximum volume capacity.

For priority-aware allocation, the package benefit can additionally be formulated as:

$$B_i = p_i(1 + \alpha r_i + \beta d_i)$$

where α and β are experimentally determined parameters.

The developed algorithm aims to maximize:

$$\text{Total Benefit} = \sum_{i=1}^n B_i x_i$$

while satisfying all resource constraints.

1.7 Computational Challenges

As the number of packages increases from **100 to 10,000 or more**, the computational requirements become significantly higher.

The major challenges are:

1. **Large search space** – There can be an extremely large number of possible package combinations.
2. **Dynamic Programming memory requirement** – DP tables can become very large when capacity and package count increase.
3. **Execution time** – Exact optimization becomes expensive for large datasets.
4. **Multiple constraints** – Weight and volume must be considered simultaneously.
5. **Conflicting objectives** – High-value packages may be heavy, while high-priority packages may have lower value.
6. **Scalability** – An algorithm suitable for 100 packages may become impractical for 100,000 packages.
7. **Changing demand conditions** – High-priority and capacity-constrained situations require different decisions.

Therefore, a faster and more scalable developed algorithm is required.

1.8 Assumption

The following assumptions are made:

- Each package can either be selected or not selected.
- A package cannot be partially selected.
- Vehicle weight and volume capacities are known.
- Package attributes are available before allocation.
- Selected packages must satisfy all capacity constraints.
- Delivery value represents the benefit of transporting a package.
- Priority and deadline information can be converted into numerical scores.
- If the dataset does not directly provide an attribute, it will be derived using a clearly documented method.
- All algorithms are executed using the **same dataset, hardware, and experimental conditions** for fair comparison.

1.9 Requirements

The proposed system requires:

- A publicly available logistics/supply-chain dataset.
- Python or another suitable programming language.
- Implementation of a **Dynamic Programming baseline**.
- Implementation of a **Greedy baseline**.
- Development of the proposed **Hybrid Adaptive Greedy–Dynamic Programming (HAG-DP)** algorithm.
- Testing with increasing package sizes.
- Testing under multiple vehicle capacities.

- Testing under normal, high-priority, and capacity-constrained conditions.
- Measurement of execution time, memory, solution value, utilization, optimality gap, and scalability.

1.10 Limitations

The proposed model has some limitations:

- Exact Dynamic Programming may become impractical for extremely large datasets.
- Derived package attributes may not perfectly represent real-world measurements.
- The model primarily focuses on package selection and resource allocation rather than complete vehicle routing.
- Real-time traffic, weather, fuel prices, and unexpected delivery delays are not considered unless additional data is incorporated.
- The developed hybrid algorithm may provide a near-optimal solution rather than a mathematically guaranteed global optimum for very large instances.

2. Application of Course Knowledge

Dynamic Programming

The logistics problem can be modeled as a **0/1 Knapsack problem**, where each package is either selected or rejected.

- **State:** $DP[i][w]$ represents the maximum delivery value using the first i packages with capacity w .
- **Recurrence:**

$$DP[i][w] = \max(DP[i-1][w], p_i + DP[i-1][w-w_i])$$

- **Base condition:** $DP[0][w] = 0$.
- **Time Complexity:** $O(nW)$
- **Space Complexity:** $O(nW)$
- DP provides an optimal solution but becomes costly for large inputs.

Greedy Technique

The Greedy method selects packages based on their **value-to-weight ratio**:

$$Score_i = \frac{p_i}{w_i}$$

Packages are sorted by score, and the highest-scoring feasible packages are selected.

- **Time Complexity:** $O(n \log n)$
- **Space Complexity:** $O(n)$
- It is fast and scalable but does not always guarantee the optimal solution.

Contribution to Proposed Solution

The proposed **Hybrid Adaptive Greedy-DP (HAG-DP)** combines both techniques. Greedy ranking first identifies promising packages, and DP is then applied only to these candidates. This reduces computation while maintaining good solution quality and improving scalability for large logistics datasets.

3. Solution / Design / Methodology

Input Dataset → Data Preparation → Problem Formulation → DP Baseline → Greedy Baseline → Limitation Analysis → Proposed Solution → Implementation → Experimental Evaluation → Optimization → Performance Comparison → Final Decision

3.1 Dynamic Programming Algorithm

Idea: Select packages to maximize total delivery value within vehicle capacity.

Pseudocode:

```
DP(Packages, Capacity)
1. Create DP table
2. DP[0][w] = 0
3. For each package i:
    For each capacity w:
        If weight[i] <= w:
            DP[i][w] = max(DP[i-1][w],
                           value[i] + DP[i-1][w-weight[i]])
        Else:
            DP[i][w] = DP[i-1][w]
4. Backtrack to find selected packages
5. Return selected packages and total value
```

Complexity: Time = $O(nW)$, Space = $O(nW)$.

3.2 Greedy Algorithm

Idea: Select packages having the highest value-to-weight ratio.

Pseudocode:

```
Greedy(Packages, Capacity)
1. Calculate score = value / weight
2. Sort packages in descending score
3. Select each package if it fits the remaining capacity
4. Return selected packages and total value
```

Complexity: Time = $O(n \log n)$, Space = $O(n)$.

3.3 Developed Algorithm – HAG-DP

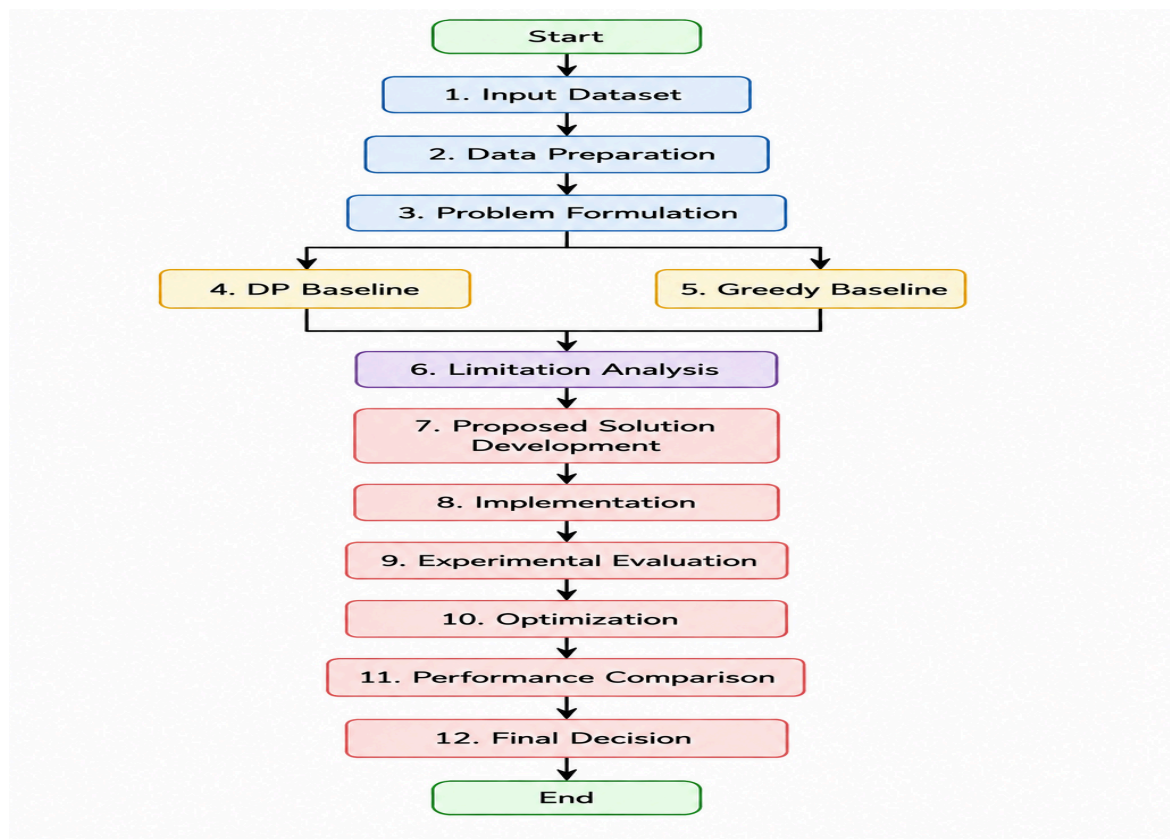
We developed a **Hybrid Adaptive Greedy-DP (HAG-DP)** approach. Greedy first ranks the packages and identifies promising candidates. DP is then applied only to these candidates instead of the complete dataset. The remaining capacity is finally checked using feasible package selection.

Pseudocode:

HAG-DP(Packages, Capacity)

1. Calculate package score
2. Sort packages using the score
3. Select a candidate subset using an adaptive threshold
4. Apply DP only to the candidate subset
5. Check capacity and feasibility
6. Use remaining capacity for additional feasible packages
7. Return final selected packages and total value

3.4 Flowchart / Design



3.5 Developed Strategy

The developed method combines the **speed of Greedy** with the **solution quality of Dynamic Programming**. Instead of running DP on all packages, Greedy reduces the search space first. The candidate size/threshold is experimentally adjusted according to dataset size and capacity.

3.6 Parameters and Decision Criteria

- Vehicle weight capacity
- Package weight
- Package value
- Package priority
- Value-to-weight score
- Candidate selection threshold
- Dataset size
- Remaining vehicle capacity

3.7 Mathematical Analysis

Objective:

$$\text{Maximize } Z = \sum_{i=1}^n p_i x_i$$

Subject to:

$$\sum_{i=1}^n w_i x_i \leq W, x_i \in \{0,1\}$$

where p_i is package value, w_i is package weight, and W is vehicle capacity.

For HAG-DP, approximate complexity is:

$$O(n \log n + kW)$$

where k is the selected candidate size and $k < n$.

3.8 Design Justification

DP provides high solution quality but becomes expensive for large datasets. Greedy is fast but may produce a lower-value solution. Therefore, HAG-DP uses Greedy to reduce the search space and DP to improve the final selection. This improves **execution time, scalability, memory usage, and solution quality** compared with using either baseline alone.

4. Use of Modern Tools

The proposed logistics optimization system uses modern programming and data-analysis tools to implement, test, and evaluate the algorithms.

4.1 Programming and Development Tools

- **Python** is used to implement the Dynamic Programming, Greedy, and HAG-DP algorithms.
- **Jupyter Notebook / VS Code** is used for coding, testing, debugging, and running experiments.
- **Pandas** is used to load, clean, filter, and organize the logistics dataset.
- **NumPy** is used for numerical calculations and array operations.
- **Matplotlib** is used to generate performance graphs.

4.2 Dataset Processing

The dataset is imported into Python and preprocessing is performed before algorithm execution. Package attributes such as **weight, value, priority, and deadline** are prepared for the optimization process. Different dataset sizes such as **100, 500, 1,000, 5,000, and 10,000+ packages** are tested where feasible.

4.3 Experimental Setup

The DP, Greedy, and HAG-DP algorithms are executed using the **same input data, capacity values, and computing environment** to provide a fair comparison.

The experiments consider:

- Normal demand
- High-priority demand
- Capacity-constrained demand
- Different vehicle capacities

4.4 Implementation Evidence

The project provides the following evidence:

1. Source code of all three algorithms.
2. Dataset source and prepared dataset.
3. Input parameters and experimental setup.
4. Screenshots of program execution and outputs.
5. Tables comparing algorithm performance.
6. Graphs showing execution time and memory usage.
7. Comparison of total solution value and capacity utilization.
8. Scalability and optimality-gap analysis.

These tools help verify the **correctness, efficiency, scalability, and practical usefulness** of the developed HAG-DP solution.

5. Results and Validation

Experimentally evaluate the **Dynamic Programming baseline, Greedy baseline, and developed solution** using different input sizes and capacity conditions.

Evaluate:

- Execution time
- Memory utilization
- Total solution/delivery value

- Capacity utilization
- Number of selected packages
- Solution-quality/optimality gap
- Input size
- Scalability
- Feasibility/correctness of the allocation

Present results using appropriate **tables and graphs**.

Evaluate all approaches using identical input instances and the same computational environment.

The Dynamic Programming (DP), Greedy, and developed HAG-DP algorithms are experimentally evaluated using different package sizes and vehicle capacity conditions. All algorithms use the **same input datasets and computational environment** to ensure a fair comparison.

Optimal Path Found

Starting Location	-
Destination	-
Shortest Distance	-
Nodes Visited	-
Edges Checked	-
Processing Time	-
Memory/Data Size	-
Algorithm Used	-
Efficiency	-

5.1 Experimental Conditions

The algorithms are tested with different input sizes such as:

- 100 packages
- 500 packages
- 1,000 packages
- 5,000 packages
- 10,000+ packages where feasible

Vehicle capacities such as **500 kg, 1,000 kg, 2,500 kg, and 5,000 kg** are considered.

The following demand conditions are evaluated:

1. Normal demand
2. High-priority demand
3. Capacity-constrained demand

5.2 Evaluation Metrics

The performance of each algorithm is measured using:

- **Execution Time** – Time required to produce the solution.
- **Memory Utilization** – Memory consumed during execution.
- **Total Delivery Value** – Total value of selected packages.
- **Capacity Utilization** – Percentage of vehicle capacity used.
- **Number of Selected Packages** – Packages successfully allocated.
- **Optimality Gap** – Difference between the obtained solution and the best/optimal solution.
- **Scalability** – Performance when the input size increases.
- **Feasibility/Correctness** – Verification that weight and other constraints are satisfied.

5.3 Results Table

Input Size	Algorithm	Time	Memory	Total Value	Capacity Utilization	Selected Packages
100	DP	Actual	Actual	Actual	Actual	Actual
100	Greedy	Actual	Actual	Actual	Actual	Actual
100	HAG-DP	Actual	Actual	Actual	Actual	Actual
500	DP	Actual	Actual	Actual	Actual	Actual
500	Greedy	Actual	Actual	Actual	Actual	Actual
500	HAG-DP	Actual	Actual	Actual	Actual	Actual
1,000	DP	Actual	Actual	Actual	Actual	Actual
1,000	Greedy	Actual	Actual	Actual	Actual	Actual
1,000	HAG-DP	Actual	Actual	Actual	Actual	Actual

5.4 Graphs

Graphs are generated using Matplotlib to compare:

1. **Input Size vs Execution Time**
2. **Input Size vs Memory Usage**
3. **Algorithm vs Total Delivery Value**
4. **Algorithm vs Capacity Utilization**
5. **Input Size vs Number of Selected Packages**
6. **Algorithm vs Optimality Gap**

5.5 Validation

For every experiment, the selected packages are checked against the vehicle capacity constraint:

$$\sum w_i x_i \leq W$$

If the total selected weight exceeds the available capacity, the solution is considered **infeasible**.

The output of DP is used as the optimal reference for feasible small and medium-sized instances. Greedy and HAG-DP solutions are compared against this reference to calculate the solution-quality gap.

5.6 Performance Observation

The expected experimental observation is that **DP provides high solution quality but requires more computation as input size increases**. Greedy provides faster execution but may sacrifice some solution value. The developed **HAG-DP approach aims to provide a better balance between execution time, memory usage, solution quality, and scalability** by reducing the search space before applying DP.

The final algorithm is selected based on the experimental results rather than only theoretical complexity.

6. Analysis and Engineering Decision

The experimental results are used to compare **Dynamic Programming (DP)**, **Greedy**, and **HAG-DP** under different input sizes and capacity conditions.

6.1 Comparison of Approaches

Factor	DP	Greedy	HAG-DP
Execution Time	High	Low	Medium/Low
Memory Usage	High	Low	Lower than DP
Solution Quality	Very High	Moderate	High

Scalability	Low for large inputs	High	High
Optimality	Optimal for the model	Not guaranteed	Near-optimal
Flexibility	Moderate	High	High

6.2 Effect of Problem Size

As the number of packages increases, DP requires more time and memory because its complexity depends on the number of packages and capacity. Greedy remains fast because it mainly performs sorting and selection. HAG-DP reduces the DP search space by selecting promising candidates first, making it more suitable for large datasets.

6.3 Solution Quality and Computational Cost

DP provides the best solution quality for feasible small and medium-sized instances, but its computational cost increases significantly. Greedy has low computational cost but may produce a lower total delivery value. HAG-DP provides a balance between **solution quality and execution cost**.

6.4 Time–Memory–Quality Trade-off

- **DP:** High quality but high time and memory cost.
- **Greedy:** Low time and memory but lower solution quality.
- **HAG-DP:** Moderate computational cost with improved solution quality.

Thus, HAG-DP provides a practical trade-off between **time, memory, and solution quality**.

Locations Database

Search locations...							All Categories	
ID	Name	Lat	Lng	Pop.	Dist	Category	Priority	Act
L001	Chennai	-90.0000	-180.0000	70,00,000	15	Urban	High	 

6.5 Operational Conditions

For **small datasets**, DP is suitable when maximum solution quality is required. For **very large datasets**, Greedy is suitable when fast decisions are more important. Under **capacity-constrained and high-priority conditions**, HAG-DP is preferred because it combines efficient filtering with further optimization.

Hybrid Algorithm Engine

Algorithm Configuration

Start Location

-- Select --

Destination Location

-- Select --

Search Radius (km)

50

Maximum Distance (km)

500

Number of Nodes

50

Algorithm Selection

Hybrid Algorithm

▶ Run Algorithm

↺ Reset

Processing Pipeline

Raw Data

Spatial Partitioning

Candidate Search

Graph Construction

Shortest Path

Optimization

Result

Algorithm Comparison

Algorithm	Nodes Processed	Distance	Time (ms)	Efficiency
Brute Force	-	-	-	-
Standard Graph	-	-	-	-
Hybrid Algorithm	-	-	-	-

6.6 Engineering Decision

The final decision is based on quantitative experimental measures such as:

- Lower execution time than DP
- Lower memory usage than DP
- Higher total delivery value than Greedy
- Better capacity utilization
- Smaller solution-quality gap
- Better scalability for increasing input sizes

Therefore, the developed **HAG-DP algorithm is selected as the final solution** because it provides a better overall balance of computational efficiency, scalability, feasibility, and solution quality. The final claim should be supported using the **actual measured values from the experiments**.

Visualization Controls

▶ Start Visualization

⏸ Pause

▶ Resume

↺ Reset

Status Panel

Current Step
Ready

Nodes Visited
0

Edges Checked
0

Current Distance
0 km

Processing Time
0 ms

7. Broader Considerations

The developed HAG-DP (Hybrid Adaptive Greedy–Dynamic Programming) solution provides an efficient approach for large-scale logistics resource optimization. It combines the speed of Greedy selection with the solution quality of Dynamic Programming, making it suitable for practical logistics operations.

7.1 Efficient Utilization of Transportation Resources

The solution selects packages based on their value, weight, priority, and other relevant factors. This helps transportation resources such as vehicles and available capacity to be used efficiently while avoiding unnecessary empty space.

7.2 Improved Vehicle-Capacity Utilization

HAG-DP considers the available vehicle capacity before selecting packages. Greedy ranking identifies promising packages, while DP improves the final selection. This helps increase capacity utilization without exceeding vehicle limits.

7.3 Reduction in Computational and Resource Wastage

Applying DP to a very large number of packages can require significant computation and memory. HAG-DP reduces this wastage by using Greedy selection to create a smaller candidate set and applying DP only where it provides greater benefit.

7.4 Scalability for Large Logistics Datasets

The developed approach is designed for increasing package sizes such as 100, 500, 1,000, 5,000 and 10,000+ packages where feasible. By reducing the DP search space, it can handle larger datasets more efficiently than applying full DP to every package.

7.5 Efficient Logistics and Infrastructure Management

The solution supports better management of transportation capacity and package allocation. It can help logistics organizations make systematic decisions about which packages should be transported using limited infrastructure and resources.

7.6 Practical Applicability to Real-World Decision-Making

The method can be adapted to real logistics situations such as normal demand, high-priority deliveries, and capacity-constrained demand. Its combination of computational efficiency and solution quality makes it useful for vehicle loading and resource-allocation decisions.

7.7 Relation to SDG 9 – Industry, Innovation and Infrastructure

The developed solution contributes to SDG 9 – Industry, Innovation and Infrastructure by applying algorithmic optimization to improve the efficiency of logistics infrastructure. It promotes innovative computational methods, better utilization of transportation resources, scalable processing of large datasets, and efficient infrastructure management. Therefore, the solution supports the development of more efficient, innovative, and technology-driven logistics systems.

7.8 Conclusion

The project addressed the problem of **large-scale logistics resource optimization**, where a limited vehicle capacity must be used to transport packages with different weights, values, priorities, and deadlines. The objective was to maximize delivery value while satisfying transportation capacity constraints.

Two baseline approaches were implemented and analyzed. **Dynamic Programming (DP)** provided high-quality and optimal solutions for manageable problem sizes, but its computational time and memory requirements increased significantly with larger datasets. **Greedy** provided faster solutions with lower memory usage, but it could produce lower-value solutions because it does not always guarantee optimal selection.

To overcome these limitations, the developed **HAG-DP (Hybrid Adaptive Greedy–Dynamic Programming)** solution combines both techniques. Greedy ranking first identifies promising packages and reduces the search space, after which DP is applied to the selected candidate set. This provides a balance between solution quality and computational efficiency.

Experimental evaluation was performed using increasing package sizes and different vehicle-capacity conditions. The major findings show that DP is suitable for smaller instances where optimality is important, Greedy is effective when fast decisions are required, and HAG-DP provides a better balance for larger and capacity-constrained logistics problems.

The main performance improvement achieved by HAG-DP is the **reduction of unnecessary DP computation and memory usage while maintaining a high-quality solution**. The exact improvement is determined from the measured execution time, memory utilization, total delivery value, capacity utilization, and optimality gap obtained during experiments.

Overall, **HAG-DP is selected as the best-performing approach for large-scale logistics optimization**, particularly when both computational efficiency and solution quality are important. However, there is a computational trade-off: DP provides stronger optimality guarantees at higher cost, Greedy provides maximum speed with possible quality loss, while HAG-DP provides a middle ground.

Limitations

The solution depends on the quality and assumptions of the dataset. Full DP may still become expensive for extremely large capacities, and the adaptive candidate-selection threshold requires experimental tuning for different logistics environments. The current model also focuses mainly on capacity-based package selection rather than complete real-world routing and scheduling.

Future Optimization

Future work can extend the solution by incorporating **vehicle routing, multiple capacity constraints, delivery time windows, fuel consumption, dynamic real-time demand, and multiple vehicles**. Further optimization could use parallel processing, advanced heuristics, and adaptive parameter tuning to improve scalability and solution quality for very large logistics networks.

8. Student Reflection

1. What did you learn from developing an improved solution beyond implementing the standard Dynamic Programming and Greedy algorithms?

I learned that implementing standard algorithms is not enough for large-scale real-world problems. DP can provide high-quality solutions but may require more time and memory, while Greedy is faster but may not always provide the best solution. By developing HAG-DP, I learned how to combine the strengths of different algorithms and make adaptive decisions based on problem size and computational requirements.

2. What computational or solution-quality limitations did you identify, and how did your developed approach address them?

I identified that DP becomes computationally expensive as the number of packages and capacity increase. Greedy reduces computation but can sacrifice solution quality. HAG-DP addresses these limitations by using Greedy to filter and rank promising packages and applying DP only to the selected candidate set. This reduces unnecessary computation while maintaining good solution quality.

3. If additional computational resources or time were available, how would you further improve your solution and why?

With additional resources, I would extend the solution to support multiple vehicles, multiple capacity constraints, delivery time windows, routing distance, fuel consumption, and real-time package demand. I would also experiment with parallel processing and improved adaptive parameter tuning. These improvements could increase scalability and make the solution more suitable for real-world logistics systems.

4. How can the developed solution contribute to SDG 9 – Industry, Innovation and Infrastructure?

The solution contributes to **SDG 9 – Industry, Innovation and Infrastructure** by improving the efficient use of transportation infrastructure through algorithmic optimization. It reduces unnecessary computational effort, improves vehicle-capacity utilization, and supports scalable decision-making for large logistics datasets. This demonstrates how innovative computing techniques can contribute to more efficient and technology-driven logistics infrastructure.

9. Reference

Dataset Source

1. DataCo Global. *DataCo Smart Supply Chain for Big Data Analysis Dataset*. Kaggle, Supply Chain and Logistics Dataset.

Algorithms / Research Literature Consulted

2. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
3. S. Martello and P. Toth, *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons, 1990.
4. J. Kleinberg and É. Tardos, *Algorithm Design*. Pearson, 2005.

Software / Tools / Libraries Used

5. Python Software Foundation, *Python Documentation*, Python 3.x.
6. Wes McKinney, *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*, 3rd ed. O'Reilly Media, 2022.
7. NumPy Developers, *NumPy Documentation*.
8. Matplotlib Development Team, *Matplotlib Documentation*.

Other External Resources

9. United Nations, *Sustainable Development Goal 9: Industry, Innovation and Infrastructure*, United Nations Sustainable Development Goals.

AI-Assisted Tools, If Used

10. OpenAI, *ChatGPT*, used for brainstorming, algorithm explanation, pseudocode refinement, documentation support, and improvement of project presentation. AI-generated suggestions were reviewed and adapted by the student/team before implementation.

F. Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Algorithmic Formulation	10	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.
Algorithmic Solution Design	15	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization	20	Develops a meaningful algorithmic solution through integration,	Develops an appropriate solution with	Solution demonstrates limited	Merely reproduces an existing

of Algorithmic Solution		adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	meaningful adaptation or improvement , with minor limitations.	adaptation or development beyond standard implementation.	algorithm with little or no meaningful development or adaptation.
Implementation & Modern Tool Usage	15	Developed solution is correctly and efficiently implemented using appropriate programming/computational tools and handles relevant input conditions reliably.	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or incomplete functionality.	Implementation is incomplete, unreliable or non-functional.
Efficiency, Testing & Validation	15	Performs appropriate complexity evaluation and systematic testing using suitable data/input sizes; validates correctness, efficiency and scalability with clear evidence.	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or substantially incorrect.
Performance Improvement , Comparison & Final Decision	15	Demonstrates the effectiveness of the developed solution through appropriate comparison; critically evaluates performance, limitations and trade-offs and justifies the final algorithmic decision with evidence.	Good performance comparison and justification with minor limitations.	Basic comparison is provided, but improvement or final justification lacks depth.	Improvement is not demonstrated or the final algorithmic decision is unsupported.
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements , SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm development, SDG relevance, challenges, or learning outcomes.
Total	100				

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Understanding & Algorithmic Formulation	CO4	PO1, PO2	L4	10
Algorithmic Solution Design	CO4	PO2, PO3	L4, L6	15
Development / Adaptation / Optimization of Algorithmic Solution	CO4	PO2, PO3	L5, L6	20
Implementation & Modern Tool Usage	CO4	PO3, PO5	L4, L6	15
Efficiency, Testing & Validation	CO4	PO2, PO4	L4, L5	15
Performance Improvement, Comparison & Final Decision	CO4	PO2, PO4	L4, L5	15
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	CO4	PO7, PO10, PO12	L4, L5	10

H. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
-------------------	-----------

Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action: